

Background

- Data in a clinical trial can be collected in a format that best suits the operational needs of the sponsor.
- However, the data needs to be standardized as per SDTM standard for submission to regulatory authorities
- Date and time values are frequently collected data points in a clinical trial, and there are multiple ways in which these can be represented
- As per SDTM standard, date and time values are to be represented in ISO 8601 standard in the SDTM datasets

What is ISO 8601 standard?

ISO 8601 is an international standard covering the worldwide exchange and communication of date and time-related data.

How are date time values represented in ISO 8601 standard in SDTM?

- Date and time components are presented in a single variable of character data type
- Date and time are separated by a delimiter 'T'
- In the date component, Year, month, and day are separated by a hyphen in between
- Year is presented as 4-character wide string (YYYY) eg: 2010
- Month is presented in 2-character wide string (MM) eg: 12 (for December)
- Day is presented in 2-character wide string (DD) eg: 03 (for 3rd day of the month)
- Missing components are represented with a hyphen or right truncated
- In the time component, hour, minute and seconds are separated by a colon in between
- Hours are represented in 2-character wide string in 24 hours format eg: 22 for 10 pm
- Minutes are represented in 2-character wide string eg: 02 for 2 minutes past an hour
- Seconds are represented in 2-character wide string eg: 09 for 9 seconds

11:59:59 pm of 31st December 2010 is presented as 2010-12-31T23:59:59

What do we learn in this lesson?

- We will see how to programmatically convert the dates to ISO 8601 format

Programming flow visualization

- Extract day component and convert it into required format (3 to '03', Unk to '-' etc.)
- Extract month component and convert it into required format (Jul to '07', Unk to '-' etc.)
- Extract year component and convert it into required format (Unk to '-' etc.)
- Process time values if required into required format
- Combine the processed values into a single date/time value
- Do any post processing if required

R programming concepts covered

We will use,

- word function to extract day, month and year components from raw date variable
- as.numeric function to convert character day values to numeric values
- suppressWarnings function to suppress printing of warnings when non-numeric values exist at the time of conversion of character values to numeric values
- is.na function to check for missing values in input data
- str_pad function to convert the numeric values to zero padded character values
 - ```
str_pad(
 string,
 width,
 pad = " "
)
```
  - we need to specify the numeric variable as first argument
  - we need to specify the overall width of the character value using width= parameter
  - we need to specify the padding character using pad= paramater (in this case, we will pad with 0s)
- str\_to\_upper function to convert the text values to upper case
- case\_when function to assign values conditionally to a new variable
- str\_c function to concatenate values of different variables into a single string
  - we need to specify the delimiter to be used between values using sep= parameter
- str\_sub to extract a substring from a string
  - ```
str_sub(string, start = 1L, end = -1L)
```
 - first argument for this function is the name of the input variable
 - we need to specify the start position of the substring using start= parameter or as a second argument
 - we need to specify the end position of the substring using end= parameter or as a third argument
 - note that in SAS substr function, we specify the number of characters to be extracted but here it is the end position
 - if we do not specify end= argument, extraction will happen till the end of the string
 - we can specify negative indices as well, a value of -2 in start argument will start extracting from second character on the right
 - ```
str_sub(aestdtc, -5)
```

 looks at the last 5 characters of the string
    - second argument is the start position
    - negative sign asks to read from right
    - third argument if omitted, reads till the end of the string
  - ```
str_sub(aestdtc, end = -5)
```

, extracts from 1st character to 5th character from the end
 - 2008---- : extracts 2008
 - start argument if omitted, is defaulted to the first character in the string